Edger8r 코드 생성 도구의 구체적 동작 분석

개요

Edger8r는 Intel SGX SDK의 핵심 컴포넌트로, EDL(Enclave Definition Language) 파일을 파싱하여 Enclave와 애플리케이션 간의 인터페이스 코드를 자동 생성하는 도구입니다. OCaml로 구현되어 있으며, 컴파일러 설계 원칙을 따라 렉서(Lexer), 파서(Parser), AST(Abstract Syntax Tree), 코드 생성기(CodeGen)로 구성되어 있습니다.

1. EDL 파일 파싱 과정에서 OCaml로 구현된 파서의 구체적 동작 코드 분석

1.1 렉서(Lexer) 구현 분석

파일 위치: `sdk/edger8r/linux/Lexer.mll`

렉서는 OCaml의 `ocamllex` 도구를 사용하여 구현되었으며, EDL 파일의 텍스트를 토큰으로 분해합니다.

주요 토큰 정의:

```
(* 데이터 타입 토큰 *)
| "char" { Tchar }
| "int" { Tint }
| "void" { Tvoid }
| "size_t" { Tsizet }
```

```
(* SGX 특화 키워드 *)
| "enclave" { Tenclave }
| "trusted" { Ttrusted }
| "untrusted" { Tuntrusted }
| "public" { Tpublic }
| "transition_using_threads" { Tswitchless }

(* 속성 키워드 *)
| "in" { 포인터 방향 속성 }
| "out" { 포인터 방향 속성 }
| "string" { 문자열 속성 }
| "count" { 배열 크기 속성 }
```

토큰화 과정:

```
rule tokenize = parse
| space { tokenize lexbuf } (* 공백 무시 *)
| newline { Lexing.new_line lexbuf; tokenize lexbuf } (* 줄 번호 추적 *)
| identifier { Tidentifier(Lexing.lexeme lexbuf) } (* 식별자 추출 *)
| number { Tnumber(string_to_int32(Lexing.lexeme lexbuf)) } (* 숫자 변환 *)
| line_comment { eat_until_nl lexbuf } (* 주석 처리 *)
```

1.2 파서(Parser) 구현 분석

파일 위치: `sdk/edger8r/linux/Parser.mly`

파서는 `ocamlyacc` 를 사용하여 구현된 LALR(1) 파서로, 토큰 스트림을 AST로 변환합니다.

문법 규칙 예시:

```
(* 함수 정의 파싱 *)
func_def: all_type Tidentifier parameter_list {
{ Ast.fname = $2; Ast.rtype = $1; Ast.plist = List.rev $3 ; }
}

(* 매개변수 타입 파싱 *)
param_type: attr_block all_type {
let attr = get_param_ptr_attr $1 in
match $2 with
Ast.Ptr _ -> fun x -> Ast.PTPtr($2, get_param_ptr_attr $1)
| _ -> fun is_ary ->
if is_ary then Ast.PTPtr($2, attr)
else Ast.PTVal $2
}
```

속성 검증 로직:

```
let get_param_ptr_attr (attr_list: (string * Ast.attr_value) list) =
let check_invalid_ptr_size (pattr: Ast.ptr_attr) =
let ps = pattr.Ast.pa_size in
```

```
if ps <> Ast.empty_ptr_size && has_str_attr pattr
then failwith "size attributes are mutual exclusive with (w)string attribute"
else pattr
in
(* 포인터 방향과 user_check 속성의 상호 배타성 검사 *)
let check_ptr_dir (pattr: Ast.ptr_attr) =
if pattr.Ast.pa_direction <> Ast.PtrNoDirection && pattr.Ast.pa_chkptr = false
then failwith "pointer direction and `user_check' are mutual exclusive"
else pattr
```

1.3 AST(Abstract Syntax Tree) 구조 분석

파일 위치: `sdk/edger8r/linux/Ast.ml`

핵심 데이터 구조:

```
(* 매개변수 타입 정의 *)
type parameter_type =
| PTVal of atype (* 값으로 전달 *)
| PTPtr of atype * ptr_attr (* 주소로 전달 *)

(* 포인터 속성 정의 *)
type ptr_attr = {
pa_direction : ptr_direction; (* in/out/inout *)
```

```
pa_size : ptr_size; (* size/count 속성 *)

pa_isptr : bool; (* isptr 속성 *)

pa_isary : bool; (* isary 속성 *)

pa_isstr : bool; (* string 속성 *)

pa_iswstr : bool; (* wstring 속성 *)

pa_rdonly : bool; (* readonly 속성 *)

pa_chkptr : bool; (* user_check 속성 *)

}


(* 함수 선언 구조 *)

type func_decl = {

fname : string; (* 함수 이름 *)

rtype : atype; (* 반환 타입 *)

plist : pdecl list; (* 매개변수 목록 *)

}
```

타입 문자열 변환:

```
let rec get_tystr (ty: atype) =

match ty with

| Char sn -> (match sn with Signed -> "char" | Unsigned -> "unsigned char")

| Int ia -> Printf.sprintf "%s%sint"

(if ia.ia_signedness = Unsigned then "unsigned " else "")

(match ia.ia_shortness with IShort -> "short " | ILong -> "long " | INone -> "")
```

```
| Ptr ty -> get_tystr(ty) ^ "*"

| Foreign s -> s
```

2. 프록시 코드 생성 과정에서 trusted/untrusted 래퍼 함수의 생성 로직 추적

2.1 코드 생성 아키텍처

파일 위치: `sdk/edger8r/linux/CodeGen.ml`

생성되는 파일 구조:

```
(* 파일 이름 생성 함수 *)

let get_uheader_name (file_shortnm: string) =

!g_untrusted_dir ^ separator_str ^ file_shortnm ^ "_u.h"

let get_usource_name (file_shortnm: string) =

!g_untrusted_dir ^ separator_str ^ file_shortnm ^ "_u.c"

let get_theader_name (file_shortnm: string) =

!g_trusted_dir ^ separator_str ^ file_shortnm ^ "_t.h"

let get_tsource_name (file_shortnm: string) =

!g_trusted_dir ^ separator_str ^ file_shortnm ^ "_t.c"
```

2.2 마샬링 구조체 생성

마샬링 구조체 명명 규칙:

```
let mk_ms_struct_name (fname: string) = "ms_" ^ fname ^ "_t"

let mk_ms_member_name (pname: string) = "ms_" ^ pname

let ms_retval_name = mk_ms_member_name retval_name
```

마샬링 멤버 생성 로직:

```
let mk_ms_member_decl (pt: Ast.parameter_type) (declr: Ast.declarator) (isecall:

let qual = if is_const_ptr pt then "const " else "" in

let aty = Ast.get_param_atype pt in

let tystr = Ast.get_tystr aty in

let ptr = if is_foreign_array pt then "* " else "" in

let field = mk_ms_member_name declr.Ast.identifier in

(* 문자열 길이 변수 생성 *)

let need_str_len_var (pt: Ast.parameter_type) =

match pt with

| Ast.PTPtr(_, pa) ->

if pa.Ast.pa_isstr || pa.Ast.pa_iswstr then

match pa.Ast.pa_direction with

| Ast.PtrInOut | Ast.PtrIn -> if isecall then true else false

| _ -> false

else false

| _ -> false

in
```

```
let str_len = if need_str_len_var pt then
sprintf "\tsize_t %s_len;\n" field else ""
in
sprintf "\t%s%s%s %s%s;\n%s" qual tystr ptr field dmstr str_len
```

2.3 ECALL 테이블 생성

ECALL 테이블 구조:

```
let gen_ecall_table (tfs: Ast.trusted_func list) =
let ecall_table_name = "g_ecall_table" in
let ecall_table_size = List.length tfs in
let tbridge_names = List.map (fun (fd: Ast.func_decl) ->
mk_tbridge_name fd.Ast.fname) trusted_fds in
let priv_switchless_bits = List.map (fun (tf: Ast.trusted_func ) ->
(is_priv_ecall tf, is_switchless_ecall tf) ) tfs in
sprintf "SGX_EXTERNC const struct {\n\
\tsize_t nr_ecall;\n\
\tstruct {void* ecall_addr; uint8_t is_priv; uint8_t is_switchless;} ecall_table
} %s = {\n\
\t%d,\n\
%s};\n" ecall_table_size ecall_table_name ecall_table_size ecall_table
```

2.4 동적 진입 테이블 생성

OCALL 허용 목록 검증:

```
let gen_entry_table (ec: enclave_content) =

let get_entry_array (allowed_ecalls: string list) =

List.fold_left (fun acc name ->

acc ^ (if List.exists (fun x -> x=name) allowed_ecalls

then "1" else "0") ^ ", ") "" trusted_func_names in

sprintf "SGX_EXTERNC const struct {\n\

\tsize_t nr_ocall;\n%s\

} %s = {\n\

\t%d,\n\

%s};\n" entry_table_field dyn_entry_table_name ocall_table_size entry_table
```

3. 타입 변환 및 검증 코드에서 포인터 검사 및 버퍼 크기 검증의 구체적 구현 분석

3.1 배열을 포인터로 변환하는 로직

```
let conv_array_to_ptr (pd: Ast.pdecl): Ast.pdecl =

let (pt, declr) = pd in

let get_count_attr ilist =

(* 각 차원의 크기를 곱하여 총 원소 개수 계산 *)

Ast.ANumber (List.fold_left (fun acc i -> acc*i) 1 ilist)
```

```
in
match pt with
| Ast.PTVal _ -> (pt, declr)
| Ast.PTPtr(aty, pa) ->
if Ast.is_array declr then
let tmp_declr = { declr with Ast.array_dims = [] } in
let tmp_aty = Ast.Ptr aty in
let tmp_cnt = get_count_attr declr.Ast.array_dims in
let tmp_pa = { pa with Ast.pa_size = { Ast.empty_ptr_size with Ast.ps_count = So
in (Ast.PTPtr(tmp_aty, tmp_pa), tmp_declr)
else (pt, declr)
```

3.2 포인터 속성 검증 로직

void 포인터 크기 검사:

```
let check_void_ptr_size (pattr: Ast.ptr_attr) (identifier: string) =
if pattr.Ast.pa_chkptr && (not (has_size pattr.Ast.pa_size))
then failwithf "`%s': void pointer `%s' - buffer size unknown" fname identifier
else ()
```

문자열 포인터 타입 검사:

```
let check_string_ptr_size (atype: Ast.atype) (pattr: Ast.ptr_attr) (identifier:
```

```
if (pattr.Ast.pa_isstr) then
match atype with
| Ast.Ptr(Ast.Char(_)) -> ()
| _ -> failwithf "`%s': invalid 'string' attribute - `%s' is not char pointer."
else
if (atype <> Ast.Ptr(Ast.WChar) && pattr.Ast.pa_iswstr)
then failwithf "`%s': invalid 'wstring' attribute - `%s' is not wchar_t pointer.
```

배열 차원 검사:

```
let check_array_dims (atype: Ast.atype) (pattr: Ast.ptr_attr) (declr: Ast.declar
if Ast.is_array declr then
if has_size pattr.Ast.pa_size then
failwithf "`%s': invalid 'size' attribute - `%s' is explicitly declared array."
else if has_count pattr.Ast.pa_size then
failwithf "`%s': invalid 'count' attribute - `%s' is explicitly declared array."
else if pattr.Ast.pa_isary then
failwithf "`%s': invalid 'isary' attribute - `%s' is explicitly declared array."
```

3.3 구조체 깊은 복사 검증

```
let is_structure_deep_copy (s: Ast.struct_def) =
let is_deep_copy (pd: Ast.pdecl) =
```

```
let (pt, _) = pd in
match pt with
| Ast.PTVal _ -> false
| Ast.PTPtr (_, attr) -> if attr.Ast.pa_size = Ast.empty_ptr_size then false els
in
List.exists is_deep_copy s.Ast.smlist


let check_structure (ec: enclave_content) =
List.iter (fun (fd: Ast.func_decl) ->
List.iter (fun (pd: Ast.pdecl) ->
let (pt, _) = pd in
match pt with
| Ast.PTVal (Ast.Struct(s)) ->
if is_structure_defined s then
let (struct_def, deep_copy) = get_struct_def s in
if deep_copy then
failwithf "the structure declaration \"%s\" specifies a deep copy is expected. R
```

4. 생성된 헤더 파일에서 함수 시그니처 및 구조체 정의의 자동 생성 과정 분석

4.1 헤더 파일 구조

공통 매크로 정의:

```
let common_macros = "#include <stdlib.h> /* for size_t */\n\n\

#define SGX_CAST(type, item) ((type)(item))\n\n\

#ifdef __cplusplus\n\

extern \"C\" {\n\

#endif\n"


let header_footer = "\n#ifdef __cplusplus\n}\n#endif /* __cplusplus */\n\n#endif
```

4.2 구조체 정의 생성

구조체 선언 매크로:

```
let mk_struct_decl_with_macro (fs: string) (name: string) =

sprintf "#ifndef _%s\n#define _%s\ntypedef struct %s {\n%s} %s;\n#endif\n"

name name name fs name


let mk_union_decl_with_macro (fs: string) (name: string) =

sprintf "#ifndef _%s\n#define _%s\ntypedef union %s {\n%s} %s;\n#endif\n"

name name name fs name
```

열거형 정의 생성:

```
let mk_enum_def (e: Ast.enum_def) =

let gen_enum_ele_str (ele: Ast.enum_ele) =
```

```
let k, v = ele in
match v with
| Ast.EnumValNone -> k
| Ast.EnumVal ev -> sprintf "%s = %s" k (Ast.attr_value_to_string ev)
in
let enbody_str =
if enbody = [] then ""
else List.fold_left (fun acc ele ->
acc ^ "\t" ^ gen_enum_ele_str ele ^ ",\n") "" enbody
in
if enname = "" then sprintf "enum {\n%s};\n" enbody_str
else sprintf "typedef enum %s {\n%s} %s;\n" enname enbody_str enname
```

4.3 함수 시그니처 생성

매개변수 문자열 생성:

```
let gen_parm_str (p: Ast.pdecl) =
let (pt, (declr : Ast.declarator)) = p in
let aty = Ast.get_param_atype pt in
let str = get_typed_declr_str aty declr in
if is_const_ptr pt then "const " ^ str else str
```

```
let get_typed_declr_str (ty: Ast.atype) (declr: Ast.declarator) =
```

```
let tystr = Ast.get_tystr ty in
let dmstr = get_array_dims declr.Ast.array_dims in
sprintf "%s %s%s" tystr declr.Ast.identifier dmstr
```

배열 차원 문자열 생성:

```
let get_array_dims (ns: int list) =
let get_dim n = if n = -1 then "[]" else sprintf "[%d]" n in
if ns = [] then ""
else List.fold_left (fun acc n -> acc ^ get_dim n) "" ns
```

4.4 브리지 함수 이름 생성

명명 규칙:

```
(* Trusted 브리지 함수 이름 *)
let mk_tbridge_name (fname: string) = "sgx_" ^ fname

(* Untrusted 브리지 함수 이름 *)
let mk_ubridge_name (enclave_name: string) (funcname: string) =
sprintf "%s_%s" enclave_name funcname

(* Untrusted 브리지 프로토타입 *)
let mk_ubridge_proto (enclave_name: string) (funcname: string) =
```

```
sprintf "static sgx_status_t SGX_CDECL %s(void* %s)"

(mk_ubridge_name enclave_name funcname) ms_ptr_name
```

5. 실제 생성 코드 예시

5.1 EDL 입력 예시

```
enclave {

trusted {

public int ecall_add([in] int* a, [in] int* b, [out] int* result);

};

untrusted {

void ocall_print([in, string] const char* str);

};

};
```

5.2 생성된 마샬링 구조체

```
typedef struct ms_ecall_add_t {

int ms_retval;

int* ms_a;

int* ms_b;

int* ms_result;

} ms_ecall_add_t;
```

```
typedef struct ms_ocall_print_t {
const char* ms_str;
size_t ms_str_len;
} ms_ocall_print_t;
```

5.3 생성된 ECALL 테이블

```
SGX_EXTERNC const struct {
size_t nr_ecall;
struct {void* ecall_addr; uint8_t is_priv; uint8_t is_switchless;} ecall_table[1
} g_ecall_table = {
1,
{
{(void*)(uintptr_t)sgx_ecall_add, 0, 0},
}
};
```

결론

Edger8r는 컴파일러 설계 원칙을 충실히 따라 구현된 정교한 코드 생성 도구입니다. OCaml의 강력한 패턴 매칭과 타입 시스템을 활용하여 EDL 파일을 안전하고 효율적으로 파싱하고, 복잡한 타입 검증과 코드 생성 로직을 구현했습니다. 특히 포인터 속성 검증, 배열-포인터 변환, 구조체 깊은 복사 처리 등의 고급 기능을 통해 SGX Enclave와 애플리케이션 간의 안전한 인터페이스를 보장합니다.